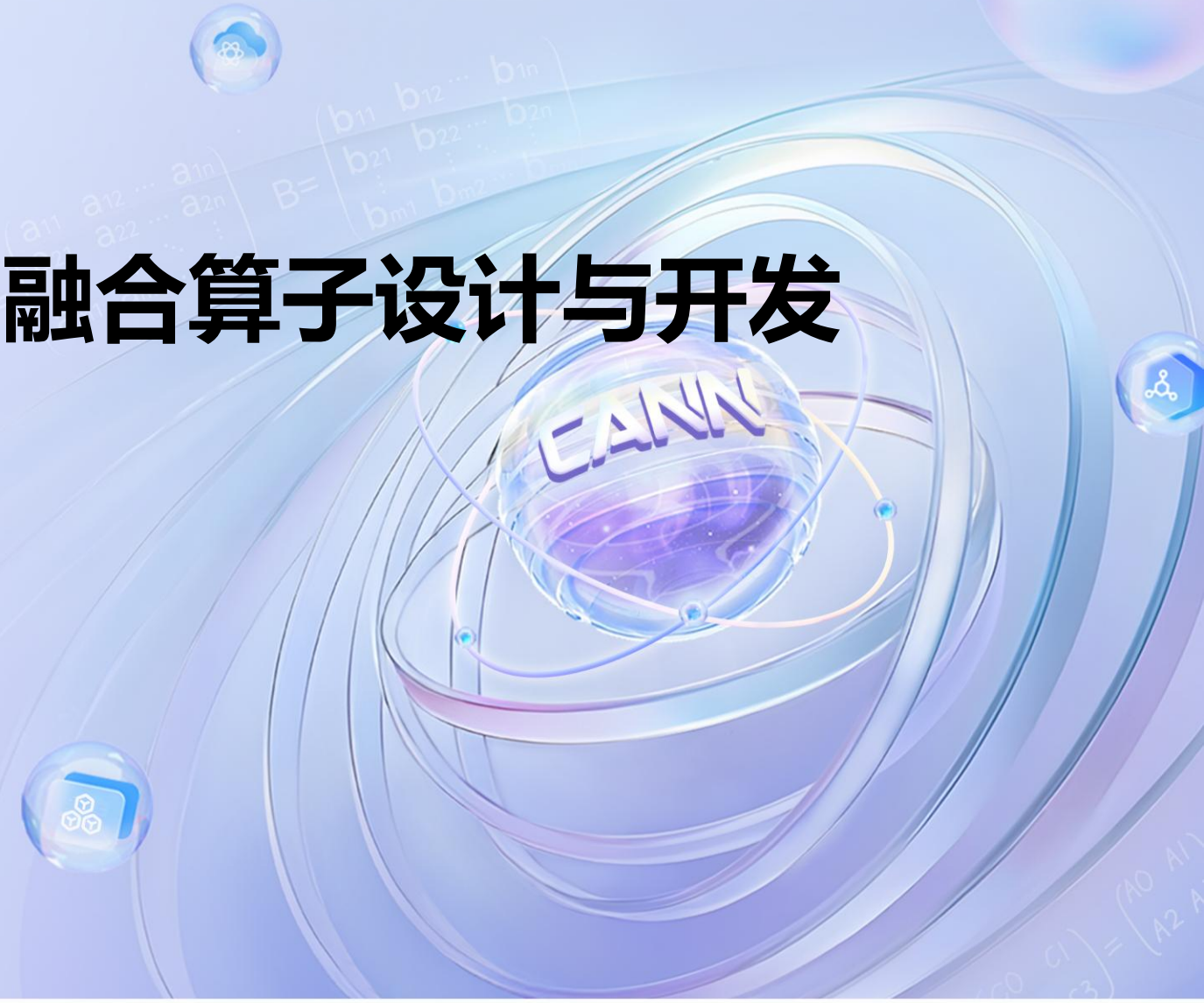
Ascend 950高性能融合算子设计与开发

讲师：王哲，史江

时间：2026.10.15



CANN

目录

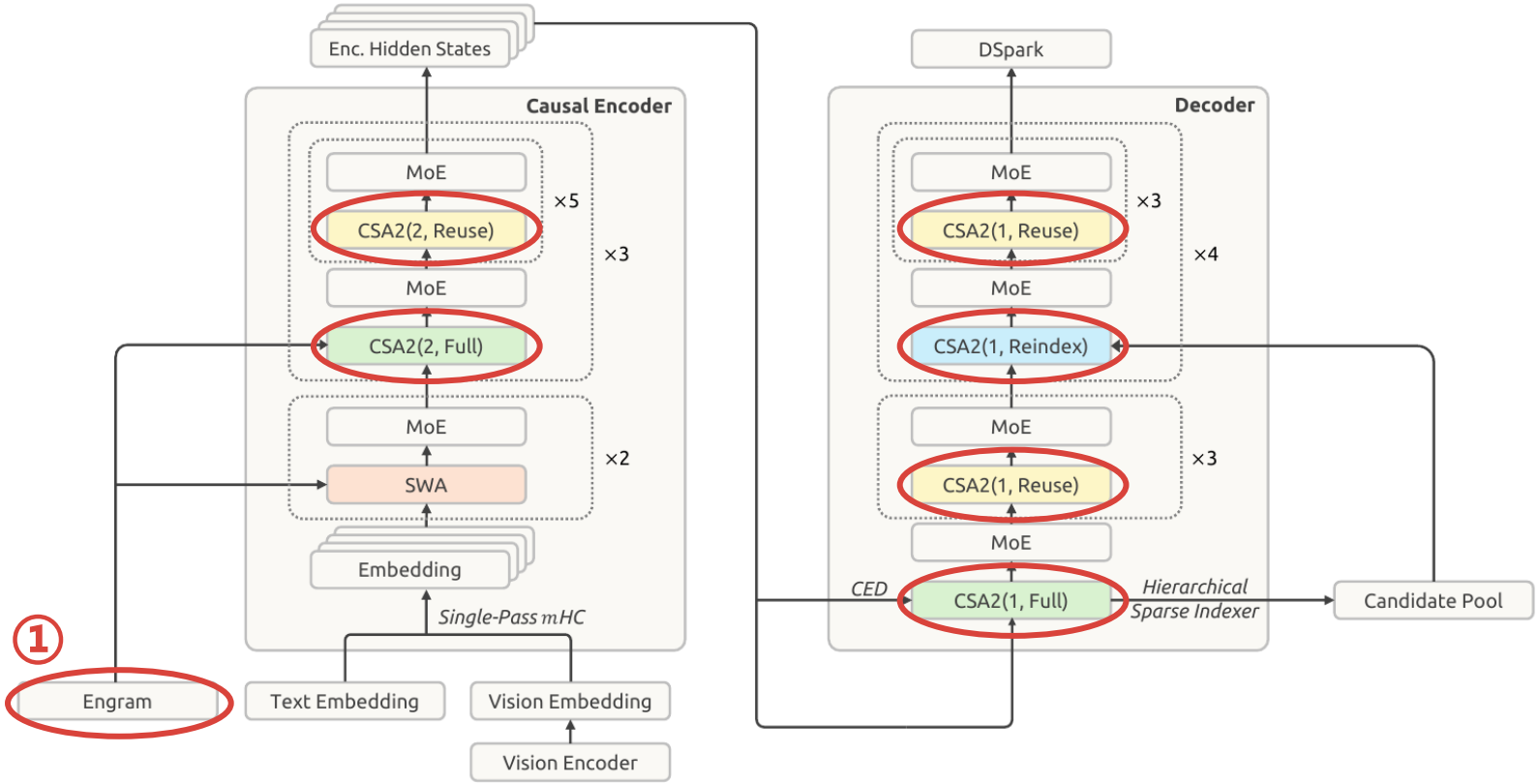
Part 1 模型与融合范围

Part 2 EngramGate的调优

Part 3 SparseFlashMLA的调优

模型总图与三种注意力模式

融合部分位于 Engram 分支和CSA2 内部



图源：DeepSeekV4.1技术报告

Engram Gate

门控记忆，更新残差

Full：生成 KV 与索引

Reindex：重选位置

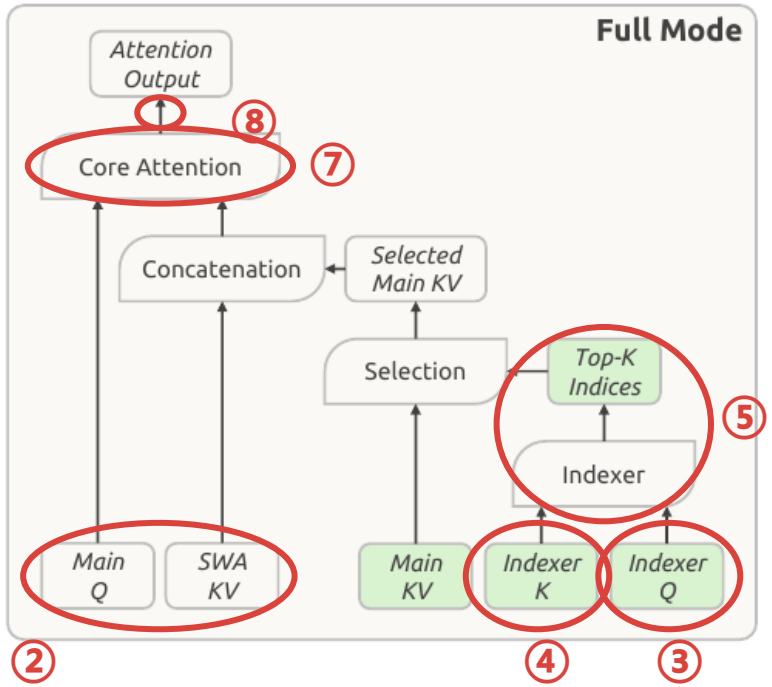
复用 KV，候选内用
全范围重选用

Reuse：复用 KV / TopK

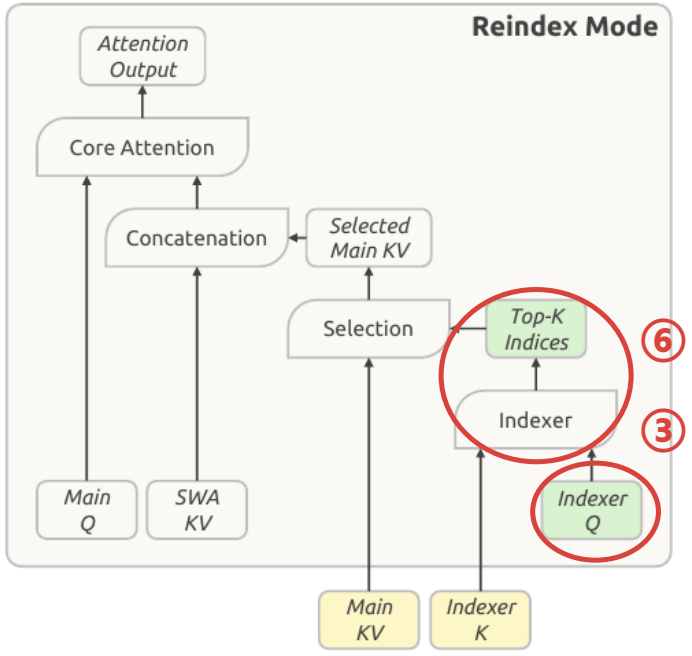
CANN

八个算子在模型局部图中的位置

Full 生成 KV 和全量索引，Reindex 在复用 KV 的基础上重选 TopK



Full：生成 Main KV 与索引 K



Reindex：复用黄色 KV / 索引 K

图源：DeepSeekV4.1技术报告

① Engram Gate
位于总图记忆分支

② Attention Prologue
Main Q 与本层 SWA KV

③ Indexer Prologue QW
索引 Q 与 head 权重

④ Indexer Prologue K
索引 K 并写入 Cache

⑤ QLI / ⑥ QSLI
全范围 / 候选池内打分选取 TopK

⑦ Sparse Flash MLA
用选中 KV 计算注意力

⑧ Attention Epilogue
变换回 Hidden 维度

https://gitcode.com/cann/cann-recipes-infer

CANN

算子融合的收益与约束

先找到真实开销，再验证数值、容量与依赖是否允许融合

原计算链的开销	融合方法	收益点	实现示例
中间量写GM后再读	算完直接交给下一步处理	减少来回搬运	AttentionEpilogue的Y量化
相邻步骤重复读输入	数据读入后继续复用	减少重复读入	SparseFlashMLA 的 K/V复用
多个小步骤分别启动	把衔接的步骤合并执行	减少启动和交接开销	EngramGate 合并计算
搬运与计算串行等待	预取与多 Buffer 轮转	减少等待时间	AttentionPrologue

融合算子的开发困境

1 模型结构变化快，融合算子定制化程度高

融合算子与网络结构紧密相关，Attention计算当前存在稠密、稀疏线性等多分支演进，而除Attention计算外，前后处理的融合算子也受cache管理、量化模式等因素影响

2 算子开发与模型迭代的矛盾

融合算子开发时间长，而当前模型迭代周期持续缩短

3 复杂的同步指令排布

融合算子计算阶段多，同步依赖相对繁琐，代码逻辑复杂

Agent开发

4 同步bug占比高且难定位

跨核同步流水链特别长，同步点距离很远

5 性能调优代码改动大

每次调优都是代码的大规模改动，很难稳定下来，效率低

模型因素

开发因素

调试因素

CANN

算子的代码更接近CPU上的表达

Attention的例子：Attention 的 QKT → Softmax → PV，Cube 和 Vector 都要用上、多Buffer，私有格式、跨核搬运、尾块等都需要开发者关注。但是通过CANNBot-DSL的表达来实现，这些细节都可以由DSL的框架（编译pass）完成。

CPU / NumPy

Attention的算法如下，没有硬件相关的信息

```
s = q @ k.T

m = s.max(axis=-1)
e = exp(s - m)
p = e / e.sum(axis=-1)

o = p @ v
```

5 行,描述的是算法

框架封装了底层机制

~~01 ping / pong 轮转~~

~~02 set_flag / wait_flag~~

~~03 Cube / Vector 拆核~~

~~04 分形 / NZ 地址~~

~~05 尾块 + 32B 对齐~~

~~06 TilingData 跨端~~

≈

算法那五行

非常接近

CANNBot-DSL

算子开发这描述数据往哪搬，然后交由编译器算地址、时序与同步等

```
nd2nz   = make_copy_engine(format_transform="nd2nz")   私有格式声明一次
fixpipe = make_copy_engine(dtype=fp32)                 L0C 搬出
s = Channel(MemLoc.UB, (BM, N), fp32, depth=2)         tile / depth 声明一次

for mi in range(ceil_div(S_q, BM)):                    尾块编译器补
    mem_copy(q_l1, q_gm[mi], engine=nd2nz)             GM → L1
    mem_copy(k_l1, k_gm,     engine=nd2nz)             分形地址不用手算
    mem_copy(l0a, q_l1); mem_copy(l0b, k_l1)           L1 → L0A / L0B
    matmul(l0c, l0a, l0b, init=True)                   Cube:S = Q·Kᵀ，同左
    mem_copy(s, l0c, engine=fixpipe)                   L0C → UB · 跨核交棒

    m = s.max(axis=-1)                                 Vector · 同左
    e = exp(s - m)                                     同左
    p = e / e.sum(axis=-1)                             同左
    mem_copy(p_l1, p); mem_copy(l0a, p_l1)             P 回 Cube
    mem_copy(l0b, v_l1, transpose=True)                转置也是声明
    matmul(l0c, l0a, l0b, init=True)                   Cube:O = P·V，同左
    mem_copy(o_gm[mi], l0c, engine=fixpipe)            UB → GM
```

多出来的 11 行,主要是声明、Tiling循环、和访存

CANN

CANNBot-DSL前端：兼顾编程易用性与极致性能表达

同一份 Python 算子：Device 实现、Host 计算与 torch 调用

Device 计算

```python
@kernel
def matmul_kernel(y: Tensor, a: Tensor, b: Tensor):
    l1_a = Channel(MemLoc.L1, (TM, KL1), dtypes.float16, depth=2)
    l1_b = Channel(MemLoc.L1, (TN, KL1), dtypes.float16, depth=2)
    l0a = Channel(MemLoc.L0A, (TM, TK), dtypes.float16, depth=2)
    l0b = Channel(MemLoc.L0B, (TN, TK), dtypes.float16, depth=2)
    l0c = Channel(MemLoc.L0C, (TM, TN), dtypes.float32, depth=2)
    cv_ub = Channel(MemLoc.UB, (TM // 2, TN), dtypes.float32, depth=2, kind=ChannelKind.CrossCore)
    out_ub = Channel(MemLoc.UB, (TM // 2, TN), dtypes.float32, depth=2)
    nd2nz = make_copy_engine(format_transform="nd2nz")
    split_m = make_copy_engine(split_axis=0)
    for m in range(get_block_idx() // 8, a.shape[0] // TM, 4):
        for n in range(get_block_idx() % 8, b.shape[0] // TN, 8):
            l0c_acc = l0c.produce()
            y_tile = tile_slice(y, (TM, TN), (m, n))
            for k1 in range(a.shape[1] // KL1):
                mem_copy(l1_a.produce(), tile_slice(a, (TM, KL1), (m, k1)), engine=nd2nz)
                mem_copy(l1_b.produce(), tile_slice(b, (TN, KL1), (n, k1)), engine=nd2nz)
                a_l1, b_l1 = l1_a.consume(), l1_b.consume()
                for k0 in range(KL1 // TK):
                    mem_copy(l0a.produce(), tile_slice(a_l1, (TM, TK), (0, k0)))
                    mem_copy(l0b.produce(), tile_slice(b_l1, (TN, TK), (0, k0)))
                    a_l0, b_l0 = l0a.consume(), l0b.consume()
                    global_k = k1 * (KL1 // TK) + k0
                    matmul(l0c_acc, a_l0, b_l0, init=(global_k == 0))
            mem_copy(cv_ub.produce(), l0c.consume(), engine=split_m)
            cv_tile, out_tile = cv_ub.consume(), out_ub.produce()
            with vf(mode="simd"):
                for row in range(TM // 2):
                    for col in range(0, TN, 64):
                        mask, _ = rr.update_mask(TN - col, elem_bits=32)
                        value = rr.vload(cv_tile, row * cv_tile.physical_stride[0] + col)
                        scaled = rr.vmuls(value, 0.5, mask=mask)
                        rr.vstore(out_tile, row * out_tile.physical_stride[0] + col, scaled, mask)
                rr.vmem_bar("vst_vld")
            mem_copy(y_tile, out_ub.consume(), engine=split_m, part_id=get_subblock_id())
```

Host 计算

```python
@host
def matmul_host(y: Tensor, a: Tensor, b: Tensor):
    matmul_kernel[32](y, a, b)
```

框架调用

```python
a = torch.randn(M, K, dtype=torch.float16)
b = torch.randn(N, K, dtype=torch.float16)
y = torch.zeros(M, N, dtype=torch.float32)
matmul_host(y, a, b)
```

易用性提升

Python 开发闭环
原生语法、编译调试、验证调用

Tensor / Tile 表达
数据计算流、切分与搬运

Cube / Vector 代码自动划分
无需显式标注，编译器自动判定归属

同步与Buffer自动管理
同步自动插入、Buffer循环复用

极致性能表达

分块与调度可控
Tiling切分、Tile块调度

高效指令可控
计算指令、搬运指令

Bank 优化可控
L1/UB Bank冲突解决

流水编排可控
计算搬运协同

目录

Part 1 模型与融合范围

Part 2 EngramGate的调优

Part 3 SparseFlashMLA的调优

EngramGate：融合范围与设计

模型定位：Engram 分支中的门控激活和残差更新

计算公式:

$$rstd = \mathrm{rsqrt}(\mathrm{mean}(x^2) + eps) * \mathrm{rsqrt}(\mathrm{mean}(key^2) + eps)$$
$$dot = \mathrm{sum}(x * weight * key) * rstd / \mathrm{sqrt}(dim)$$
$$gate = \mathrm{sigmoid}(\mathrm{copysign}(\mathrm{sqrt}(\mathrm{max}(\mathrm{abs}(dot), clamp_value)), dot))$$
$$y = \mathrm{bf16}(x + gate * value)$$



mask 选择输出路径；查表与投影位于算子上游，不在 Engram Gate 内融合。

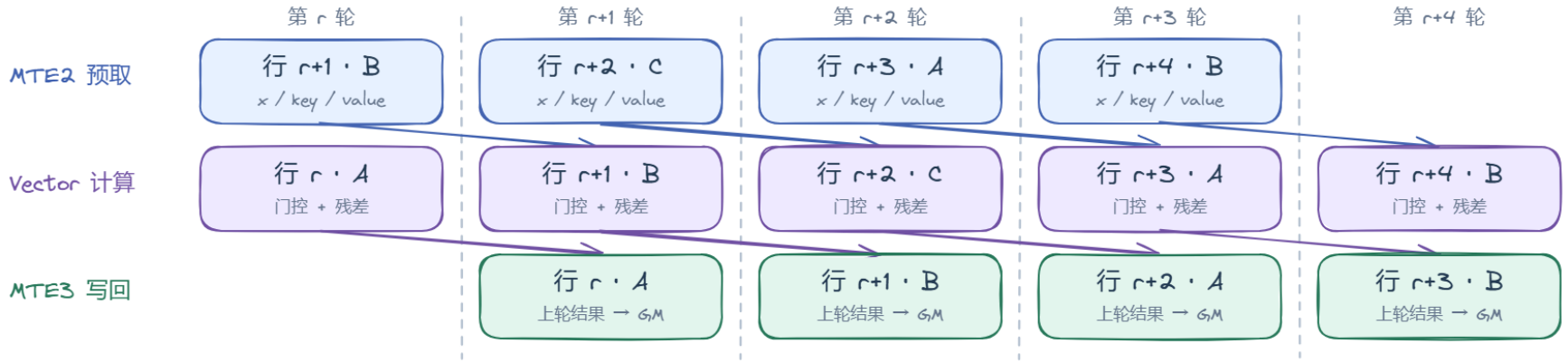
EngramGate：DSL 开发、流水调优与实测

T=72、H=4、D=5120，28核的测试环境

DSL：显式 UB 分配与切片搬运

```
self.x_a = Buffer(
    MemLoc.UB, (self.dim,),
    dtypes.bfloat16
)
mem_copy(self.x_a,
    gm_x[first_token, hc_idx, None])
```

任务与存储	关键设计
288 行 / 56 核	每核 5–6 行，固定 HC 权重驻留
UB： 150 kB	输入三缓冲，输出双缓冲，2 份 FP32
5120 / 64 = 80 个向量段	三条累加链，行末三次 FP32 归约



T72 / 阶段	原始	跨行预取	输出双缓冲	输入三缓冲
时长 / μs	14.331	7.893	7.090	6.5975

目录

Part 1 模型与融合范围

Part 2 EngramGate的调优

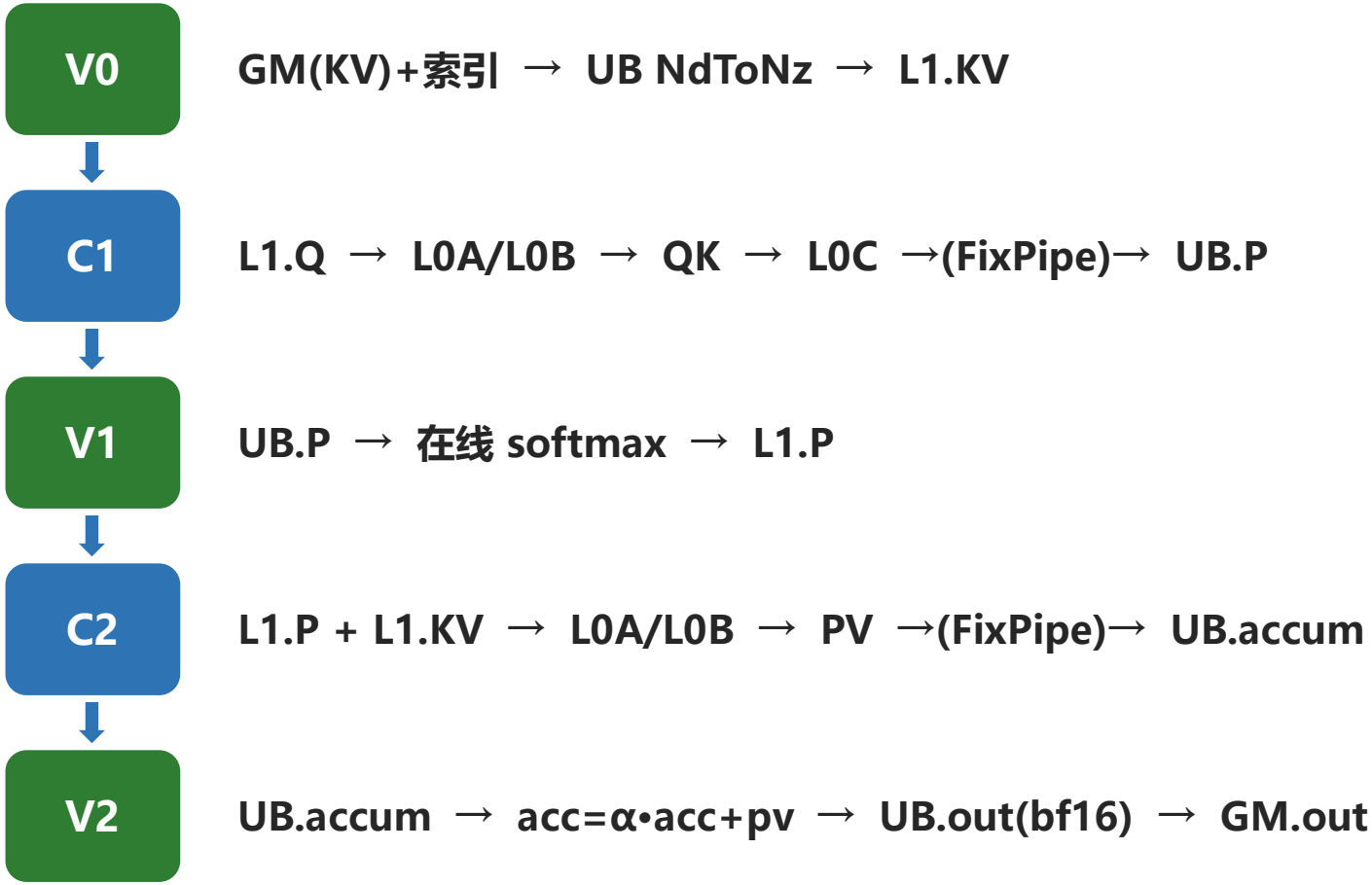
Part 3 SparseFlashMLA的调优

SparseFlashMLA：计算流程

算子旨在完成以下形式的 Attention 计算，其中，$\tilde{K} = \tilde{V}$是根据topk index筛选出来的。

$$O = \text{softmax}(Q@\tilde{K}^T \cdot \text{softmax_scale})@\tilde{V}$$

计算核间切块64*128，包含下面5个计算阶段



SparseFlashMLA：Cube计算内存精细排布

- C侧流水排布：QK(0) | {QK(k+1) PV(k)} … QK(k+1) PV(k) | PV(last)



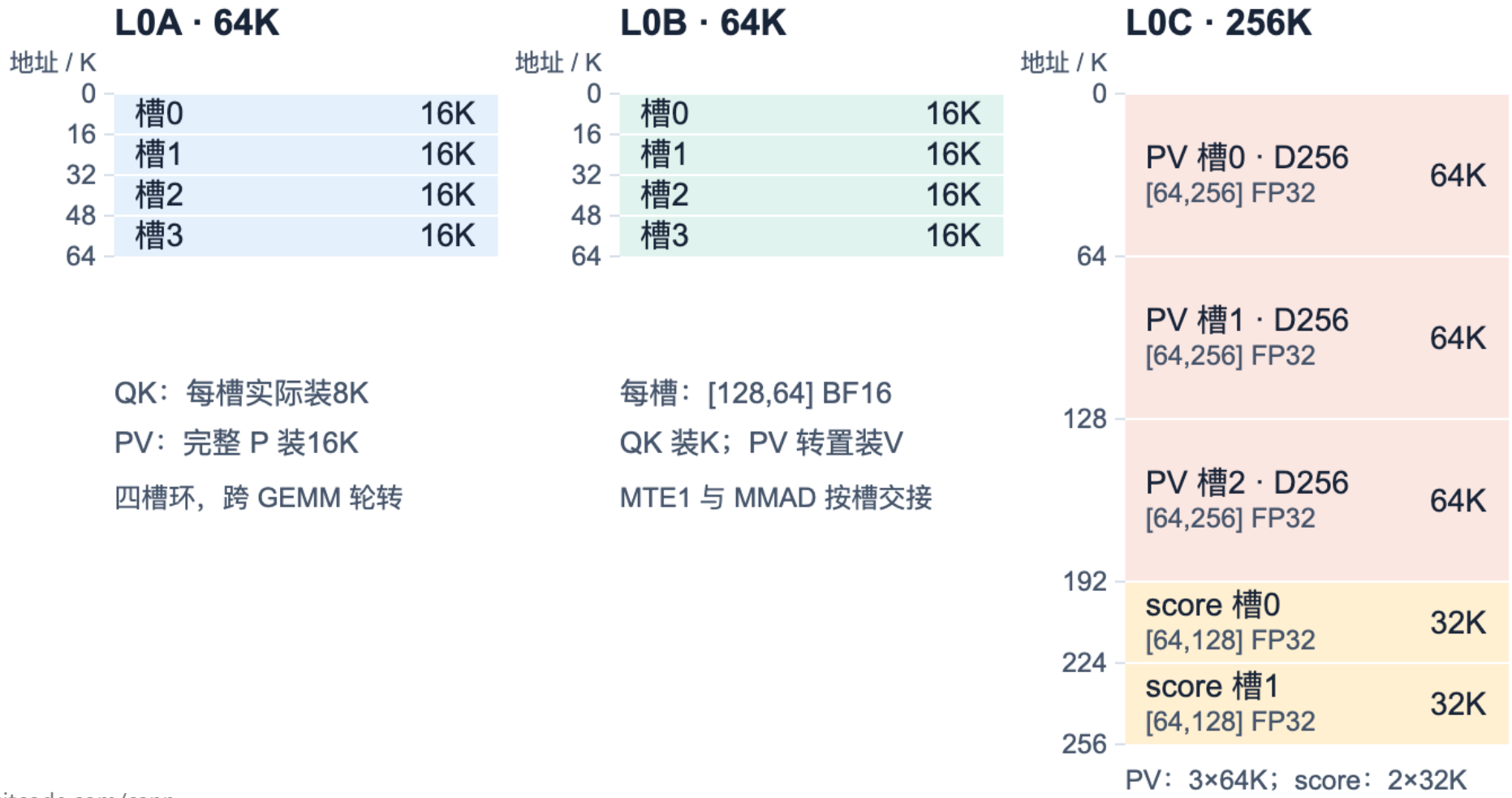
- **C1=QK**
- 左矩阵Q：64*512，右矩阵K：128*512
- **Q矩阵L1空间**：64*512的数据量拆分成(32*384+32*128)*2，其中32*384*2开DB，buffer轮转使用，32*128*2部分单buffer，不复用
- **K矩阵L1空间**：128*512的数据量拆分成64*512*2，开3buffer
- M = 64，N = 128（KV_S轴），K = 64(D轴），8次MMAD完成整体计算

- **C2 = PV**
- 左矩阵P：64*128（单buffer），右矩阵V：128*512（与K相同）
- M = 64，N = 64（D轴），K = 128(KV_S轴），8次MMAD完成整体计算
- 输出沿D轴切256，完成64*256的计算就触发提前启动V2计算。

CANN

SparseFlashMLA：Cube计算内存精细排布

L0 · 四槽输入环与 L0C 多缓冲



QK：每槽实际装8K

PV：完整 P 装16K

四槽环，跨 GEMM 轮转

每槽：[128,64] BF16

QK 装K；PV 转置装V

MTE1 与 MMAD 按槽交接

CANN

SparseFlashMLA：Vector计算多级流水排布

Tling切块：核间基本块64*128，由于C:V=1:2，所以每个Vector的基本块大小是32*128



序号	实现选择	start	作用
1	prefetch_issue_index(i)	0	GM->UB，拷贝当前基本块需要的index（128个token）
2	v0(i+1)	1	根据topk完成gather kv，完成nd2nz转换，发起UB->L1拷贝，每次处理32个token
3	v1(i+3)	3	在线softmax计算
4	v0(i+1)	1	根据topk完成gather kv，完成nd2nz转换，发起UB->L1拷贝，每次处理32个token
5	v2(i+4)	4	（1）对V2结果完成rescale，每次处理D512的数据 （2）与C2按D256切分。C2阶段每完成D256的计算就触发Fixpipe，V2等到D256就启动rescale计算。

CANN

SparseFlashMLA：Vector计算多级流水排布



- Tling切块：核间基本块64*128，表示对kv的切块是128。
- 由于C:V=1:2，在V0阶段，每个vector负责64个token的处理。
- 考虑负载均衡，以及UB内存限制，每个V0阶段处理32个token，穿插在V1/V2操作中间。

UB	数据类型	用途
topk_indices[2][128]	int32	GM->UB，拷贝当前基本块需要的index（128个token），开2 Buffer
gathered_kv[2][32][512]	bf16	根据topk完成对应kv数据的GM->UB拷贝，每次处理32个token，ND存储，开2 buffer。
gathered_kv_nz[33*512]	bf16	通过RegBase VF编程，实现nd2nz格式转换，NZ存储，单buffer。

CANN

SparseFlashMLA：Vector计算多级流水排布

④ softmax

UB	数据类型	用途
c1_res[32*128]	fp32	保存C1结果，通过Fixpipe写入，ND格式，单buffer
p_out[33*128]	bf16	保存softmax结果，NZ格式，单buffer
softmax_max[2][64]	fp32	保存在线softmax的max值，开2 buffer
softmax_sum[2][64]	fp32	保存在线softmax的sum值，开2 buffer
softmax_exp[2][64]	fp32	保存在线softmax的exp值，开2 buffer

⑤输出 merge

UB	数据类型	用途
c2_res[32][512]	fp32	保存C2结果，通过Fixpipe写入，ND格式，单buffer
v2_res[32][512]	fp32	V2 rescale结果常驻UB，ND格式，单buffer

CANN

SparseFlashMLA：性能数据

Prefill典型场景测试：

Q_S	KV_S	TOPK	DSL (us)	MFU
4096	4096	640	813.007	97.71%
4096	8192	640	813.174	97.69%
4096	32768	640	827.885	95.95%

Q_S线性度测试：

Q_S	KV_S	TOPK	DSL (us)	MFU
512	10240	640	115.085	86.28%
1024	10240	640	215.101	92.33%
1536	10240	640	315.261	94.49%
2048	10240	640	415.206	95.66%
2560	10240	640	515.223	96.36%
3072	10240	640	615.597	96.78%
3584	10240	640	716.043	97.07%
4096	10240	640	816.870	97.25%

CANN

Ascend 950 高性能融合算子



技术报告地址：　https://gitcode.com/cann/cann-recipes-infer

代码仓地址：　https://gitcode.com/cann/cannbot-dsl

欢迎前往仓库查阅体验！

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann